

ANÁLISIS TÉCNICO ESTRUCTURAL Y ALGORÍTMICO

Evaluación del Sistema de Optimización de Rutas y Transporte de Paquetes

Este documento presenta un análisis analítico riguroso del software desarrollado para la gestión de redes de transporte, detallando las decisiones de diseño arquitectónico, las estructuras de datos seleccionadas y las complejidades asintóticas asociadas a sus operaciones fundamentales.

1. Estructuras de Datos Utilizadas y Justificación

Para garantizar una manipulación eficiente del modelo abstracto de la red de transporte, se han integrado las siguientes estructuras de datos organizadas bajo el paradigma de Programación Orientada a Objetos (POO):

- **Diccionario de Diccionarios (Lista de Adyacencia):** El grafo se modela internamente utilizando un *dict* nativo de Python donde las claves representan los nodos de origen, y sus valores asociados son otros diccionarios que contienen pares *{nodo_destino: peso}*. Esta elección proporciona accesos, inserciones y actualizaciones en tiempo constante promedio $O(1)$, superando significativamente el rendimiento de las matrices de adyacencia en grafos dispersos.
- **Min-Heap / Cola de Prioridad (Módulo *heapq*):** En el algoritmo de Dijkstra se utiliza un montículo binario para gestionar las distancias tentativas. Almacena tuplas estructuradas como *(distancia_acumulada, nodo)*. Esto permite extraer de manera eficiente el elemento con la menor prioridad (menor coste) en tiempo logarítmico, acelerando la convergencia del algoritmo.
- **Conjuntos (Sets):** Tanto en la búsqueda en anchura (BFS) como en Dijkstra, se emplean objetos de tipo *set* para registrar las ubicaciones ya procesadas (nodos visitados). Dado que las búsquedas en un conjunto se basan en tablas hash, la verificación de existencia se ejecuta en $O(1)$, lo que mitiga el riesgo de bucles infinitos en grafos con topologías cíclicas.
- **Cola Doblemente Terminada (*collections.deque*):** Para la verificación de conectividad mediante BFS, se utiliza una cola estructurada como FIFO. La implementación con *deque* asegura que las operaciones de encolar (*append*) y desencolar por el extremo opuesto (*popleft*) se realicen estrictamente en $O(1)$, a diferencia de las listas tradicionales que penalizan los desplazamientos de memoria con $O(V)$.

2. Análisis de Complejidad Temporal

A continuación se evalúa la eficiencia temporal de las tres funciones principales del sistema, expresadas mediante la notación asintótica Big O:

Operación / Algoritmo	Complejidad Temporal	Justificación Teórica
Añadir Conexión (agregar_arista)	$O(1)$	La inserción o actualización de un par clave-valor en la lista de adyacencia se realiza mediante direccionamiento directo en la tabla hash interna de Python. El coste es constante independientemente del tamaño de la red.
Ruta Más Corta (Dijkstra)	$O((V + E) \log V)$	Cada uno de los V vértices se extrae del Min-Heap una sola vez ($O(V \log V)$). Para cada una de las E aristas, se realiza un proceso de relajación que puede actualizar y empujar un nuevo elemento al montículo ($O(E \log V)$).
Comprobación de Conexión (BFS)	$O(V + E)$	En el peor caso (nodos interconectados o exhaustivos), el algoritmo añade y extrae de la cola FIFO cada vértice exactamente una vez, y examina todas las aristas Incidentes de la lista de adyacencia.

Donde V representa el número total de vértices (ubicaciones/estaciones) y E representa el número total de aristas (rutas/enlaces directos) existentes en el grafo.

3. Complejidad Espacial del Grafo

La complejidad espacial de la estructura de datos que almacena el grafo es de $O(V + E)$.

Esta cota es óptima para grafos dispersos debido a las siguientes razones:

- El diccionario base contiene exactamente V claves independientes, una por cada estación o ubicación registrada en el sistema.
- Dado que implementamos un grafo no dirigido (bidireccional), cada ruta o enlace entre dos puntos genera dos entradas en las listas de adyacencia internas de los respectivos nodos vecinos. Por lo tanto, el número total de subelementos almacenados en memoria escala de forma estrictamente lineal con respecto a $2 \times E$.

4. Áreas de Mejora y Evolución del Sistema

A pesar de cumplir rigurosamente con los requisitos operacionales, la arquitectura de software actual presenta oportunidades significativas de optimización técnica:

A. Migración a una Base de Datos Relacional (SQLite)

Actualmente, la persistencia se apoya en la serialización completa de archivos JSON. Esto obliga al programa a cargar la red entera en la memoria RAM del sistema. Reemplazarlo por un módulo de base de datos SQL local permitiría ejecutar consultas indexadas bajo demanda, reduciendo la huella de memoria en redes de gran escala.

B. Validación Avanzada de Rutas Multicapa

La lógica de negocio actual sobrescribe el peso de una arista existente si se añade de nuevo. Una versión industrial debería admitir aristas paralelas (múltiples rutas con diferentes costes entre los mismos destinos) o implementar validaciones defensivas automáticas para conservar siempre el camino óptimo.

C. Integración de Benchmarking y Profiling Empírico

Añadir al menú interactivo funciones analíticas que utilicen herramientas estándar como ***time*** o ***timeit*** facilitaría la captura automática de tiempos de respuesta en milisegundos. Esto permitiría al usuario contrastar el rendimiento real empírico frente a las cotas teóricas asintóticas calculadas en este reporte.